

Nape replica — milestones

A hand-written, **bit-exact** TypeScript replica of the **original game's** Nape physics engine, built to eventually replace the Haxe-compiled `public/assets/nape.js` in this project. Lives in `src/physics/replica/`.

⚠ The version rule (read first)

We convert and test against **one** Nape version only: the one in the **actual game** — its own engine decompiled from the shipped `SoccerBalls2.swf`, at `tools/swf-decomp/scripts/zpp_nape/*`.

`public/assets/nape.js` is **NOT the reference**. It is a different, newer Nape build (nape-haxe4 2.0.22) whose integrator was refactored — e.g. the original uses *force-based* drag (`force += vel · (-drag · mass)`) while nape.js uses *multiplicative* drag (`vel = (1-dt · drag) · vel`); they diverge in the bits once a body has horizontal motion. Several Nape versions exist in this repo; only the game-SWF decompile counts. (nape.js still ships as the live runtime engine until the M7 cutover — it just isn't what the replica is graded against.)

Why

Physics-feel is the project's top risk. The compiled Haxe engine is already proven faithful to the shipped game, so the replica's payoff is **transparency, debuggability, and dropping the Haxe toolchain** — a readable engine we own and can step through.

The rule: bit-identical or it isn't done

Each milestone is "done" only when its differential test is **bit-for-bit identical** to the reference engine — not epsilon-close. Comparisons use `Object.is` (so `-0 ≠ 0`, `NaN = NaN`), which is exact IEEE-754 identity for the doubles JS produces. We achieve this by porting Nape's arithmetic *literally* (same operation order, no reassociating float expressions), since two double computations are bit-equal only when their operation order matches.

How it's tested — against the real original AS3

The oracle is the **original Nape AS3 itself, executed** — not a transpile (a hand-transpile would just be another guess). The pipeline:

1. A tiny **calling-AS3 harness** (`tools/nape-oracle/harness-mN.as`) builds the milestone's scenario using the real Nape API and `trace()` s each value as raw IEEE-754 bits (via `ByteArray`, so no decimal loss). It replaces the SWF's document class (`Preloader`), so the SWF boots straight into the physics test — **no game, no levels, no rendering**.
2. **FFDec** injects it into a copy of the game SWF, which already contains the real original Nape bytecode (`- replace ... Preloader harness.as`).
3. **Ruffle** runs it headless (driven by Puppeteer); the `trace()` d bits are captured into a committed fixture `original-goldens/mN-*.json`.
4. The **vitest gate** asserts the ported replica reproduces those bits exactly.

Capture-once → commit → fast test: the truth is the real original AS3, but the dev loop stays quick.

Float caveat: the replica runs in V8, the original in AVM2. For `+` `-` `*` `÷` `√` they're bit-identical; `sin/cos` may differ by ≤ 1 ULP (and Flash's trig was OS-dependent). M0 free-fall has no trig, so it's exact — watch this from M1 (rotation) on.

Pieces:

File	Role
<code>tools/nape-oracle/harness-mN.as</code>	calling AS3 — drives real Nape, traces bits
<code>tools/nape-oracle/capture-golden.mjs</code>	FFDec-injected SWF → Ruffle/Puppeteer → fixture
<code>tools/nape-oracle/capture.mjs</code>	generic Ruffle console capture (debug)
<code>src/physics/replica/original-goldens/*.json</code>	committed real-original bit goldens
<code>nape-core.ts</code>	<code>NapeReplica</code> — the readable ported engine (the deliverable)
<code>diff.ts</code>	<code>f64hex</code> , <code>bitEq</code> — bit-exact helpers
<code>mN-*.test.ts</code>	per-milestone gates (replica vs golden)

How to run

```
# all replica gates
npx vitest run src/physics/replica

# one milestone gate
npx vitest run src/physics/replica/m0-freefall.test.ts

# watch mode while developing a milestone
npx vitest watch src/physics/replica

# the whole project test suite (includes these)
npm test
```

Regenerate a golden from the real original AS3 (only when the harness changes):

```
# 1. inject the calling-AS3 harness into a copy of the game SWF
java -Djava.awt.headless=true -jar tools/vendor/ffdec.jar -replace \
  /Users/jonscott/Projects/SoccerBalls2/bin/SoccerBalls2.swf \
  tools/nape-oracle/m0-oracle.swf Preloader tools/nape-oracle/harness-m0.as

# 2. run it under Ruffle headless and capture the bit-golden
node tools/nape-oracle/capture-golden.mjs tools/nape-oracle/m0-oracle.swf \
  src/physics/replica/original-goldens/m0-freefall.json
```

Roadmap

#	Milestone	Scope	Status
M0	Free-fall	circle mass + <code>updateVel</code> / <code>updatePos</code> integration	✅ done
M1	Rotation + polygons	angular dynamics, <code>applyImpulse</code> , polygon mass props	🕒 planned
M2	Broadphase	pair generation + ordering	🕒 planned
M3	Narrowphase	contact manifolds (circle/circle, circle/poly, poly/poly)	🕒 planned
M4	Solver	arbiter / sequential-impulse (the load-bearing 80%)	🕒 planned
M5	Joints	rev (pivot+motor+angle), weld, distance	🕒 planned

#	Milestone	Scope	Status
M6	Ray + sleeping	raycast, islands/sleeping, <code>wakeJointPartners</code>	 planned
M7	Cutover	drop <code>nape.js</code> as a runtime dependency	 planned

Convex decomposition (`GeomPoly.convexDecomposition`) is deliberately kept **out** of the bit-exact loop: we precompute sub-polygons once and feed identical pieces to both engines.